

Serialization for the Parking System

Software Design & Programming

Object-Oriented Programming II

Joseph DiBiasi

University of Denver College of Professional Studies

5/23/2026

Faculty: Nathan Braun, MBA

Director: Cathie Wilson, MS

Dean: Bobbie Kite, MLS

Introduction

The university's parking application has grown significantly in features and organization since its inception, yet it has lacked the ability to serialize data. This development cycle we have aimed at providing proof of concept for the execution of the commands we implemented previously. We aim to demonstrate successful communication from a test client to our backend server. This communication will demonstrate serialization by transferring user input on the client side into a GSON object that will then be transmitted to the server where it will be deconstructed to be used for the execution of commands. This simple workflow will be the proof of concept that will eventually expand into a complex command structure capable of meeting all our needs.

Requirements

Demonstrating this initial workflow will require us to mockup a simple server that can be run and accessible via Terminal. We will also require a server client that can represent our simple front-end. This server client needs to be capable of accepting user input then serializing that input and transmitting it to the back-end server. The back-end server then needs to deserialize that data and use it in the intended workflows. Eventually, this will encompass a large number of commands. For now, we are only supporting the commands pertaining to the registration of customers and their cars. Customers will need to be registered in the session before registering a car. Once the workflow is finished, the back-end server will need to serialize the response, returning both a status code and results of the attempted action to the server client.

High-Level Design

We want our process to flow smoothly to demonstrate our initial command workflow, that means we are not going overboard on the server-client structure. We have a simple server that will listen for commands when it is running. When the server is running, we have a server client that can be targeted with commands which it will transmit to the back-end server. A parking request data transfer object (DTO) is used to send these client commands. The server returns a response through the parking response DTO; the response includes the status code as well as the results of the attempted command request. Commands that are not recognized, as well as the command 'list', will result in the back-end server returning a status for a bad request as well as the currently registered commands. Commands that are recognized will be validated then executed if validation passes. We are not storing data long-term for this first iteration; this means each working session will require an initial customer registration. Successful customer registration will return a customer id that can be used to register a car in the same session.

```
PS C:\Users\jjdib\Workspace\ICT_4315\parking_system> java -cp target/classes clients.ServerClient CUSTOMER name=Joseph address=aurora phonenum=3031234567
::: Command ::: CUSTOMER CUSTOMER, name=Joseph, address=aurora, phonenum=3031234567
May 24, 2026 12:27:37 PM clients.ServerClient runCommand
INFO: You are now connected to: 127.0.0.1
Customer registered successfully:
Name: Joseph
CustomerID: 7c2b115d-e3b2-4159-bda4-a4a35b89c2d1
Address: aurora
Phone Number: 3031234567
```

Figure 1. Successful Execution of the Customer Command

```
PS C:\Users\jjdib\Workspace\ICT_4315\parking_system> java -cp target/classes clients.ServerClient car license=SWEET cartype=SUV ownerid=7c2b115d-e3b2-4159-bda4-a4a35b89c2d1
::: Command ::: CAR car, license=SWEET, cartype=SUV, ownerid=7c2b115d-e3b2-4159-bda4-a4a35b89c2d1
May 24, 2026 12:29:08 PM clients.ServerClient runCommand
INFO: You are now connected to: 127.0.0.1
Car registered successfully:
Owner ID: 7c2b115d-e3b2-4159-bda4-a4a35b89c2d1
License: SWEET
Car Type: SUV
```

Figure 2. Successful Execution of the Car Command

Implementation

The actual implementation of this design was fairly straightforward once the initial defects were ironed out. This experience essentially lined up with our goal of proof of concept. We had a little difficulty in initially implementing communication between the client and server. There were problems bringing in the needed dependencies and ensuring consistency in testing. Modifications were made to the pom to better utilize the dependencies, and a server startup script was created to make the startup process a little smoother. Once that initial client-server connection was made, work proceeded smoothly. We were able to implement the two pre-existing commands regarding the registration of both customers and their cars with only minimal changes to their structure to support the new parking request and response DTOs. This means that there is some validation present in these command workflows, but they are still very much operating at a limited level for demonstration purposes.

Future development will include a serious elevation of checks in the parking system when executing commands sent by the client. We want to be sure that our workflows will be consistent and our responses valuable to both customers and developers. To that end, the list command and invalid command workflows will be eventually separated. While we still want to give customers the ability to list out registered commands and provide guidance when they enter an incorrect command, it is harmful to our troubleshooting to not be able to differentiate between the two in a response. Later updates will see the list command officially registered and only invalid entries triggering a bad request status code. The command registration process itself will also be reworked. While we intended to make this into a more dynamic feature, unexpected delays in achieving initial client to server communication meant that commands are

currently hardcoded to be registered on server startup. All current features did receive updates to their test cases to reflect the new workflow while new cases were added to cover the additional classes. For full test suite execution, the server should not be active. Certain tests will be skipped if the server is running due to listening on the same port.

Summary

The initial concept of communication between a client and server to execute commands was our goal; we achieved it. The growing backlog of future development work is not a black mark but one denoting how much progress we have made. Our parking system application has grown so much that the potential development possibilities are starting to grow exponentially. We have laid the groundwork for an application that is ready to respond dynamically to the different demands that will be made on our parking office. Demonstrating the ability to serialize data, transmit it, then deserialize it for consumption is the backbone of a fully-fledged development environment. This gives us the opportunity to store data long-term, access it at will, and transmit it where needed. The future of our parking system development can take many different paths; by demonstrating a strong fundamental serialization method, we are up to the challenge no what which path we choose to pursue next.